Wearable Gesture Controller

An IMU-Based Edge Device for Gesture-Driven Smart Home Control

CPU-only demonstration • No hardware required • Synthetic data

Arya Choudhary
240911438

Sanika Singh
240911664

Vishad Sharma
240911580

V Semester B.Tech. IT · School of Computer Engineering · MIT Manipal

Problem statement and motivation

An additional interaction channel

Speech, reach, or fine-motor limitations can make conventional controls difficult to use.

The engineering problem

Translate a bounded six-channel motion window into a command, while allowing NO_COMMAND.

Accessibility benefit requires future user evaluation.

Why an inertial interface?

- Acceleration + angular velocity
- Local temporal recognition
- No camera or internet dependency
- Background and uncertain inputs need rejection

Literature gap and objectives

What the recent literature supports

Wearable models: preserve time and channel structure.

Embedded ML: budget activations and kernels, not only weights.

Benchmarking: separate quality, latency, memory, and energy.

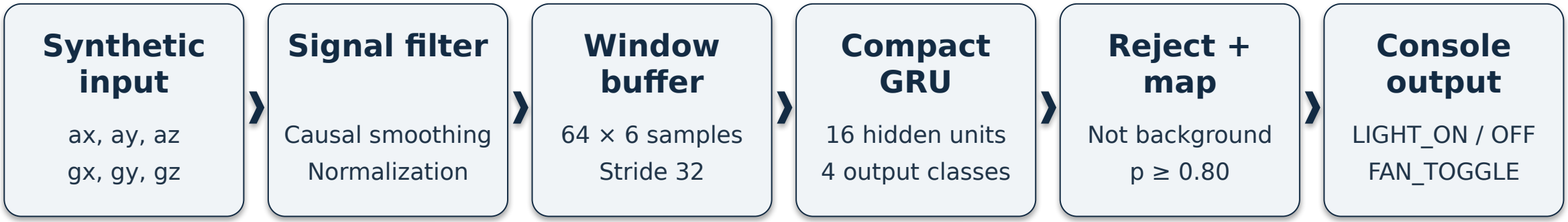
Our application-specific gap

An inspectable six-channel → GRU → reject → command pipeline, with reproducible evidence.

1. Keep recordings separate across splits
2. Verify preprocessing and prediction
3. Expose NO_COMMAND
4. Bound embedded performance claims

10 recent papers connect wearable recognition, efficient deployment, and embedded evaluation.

Sensor-to-command architecture



Future physical path — not implemented

IMU + MCU sampling → supported inference runtime → optional BLE / Wi-Fi receiver → appliance

No MCU, board, radio, receiver, or appliance is needed for the current software.

Methodology and model choice

96 recordings
480 training windows

32 recordings
160 validation windows

40 recordings
200 test windows

Split independent recordings BEFORE windowing.

Filter → training-only normalization → 64-sample windows.

Seed 42; five windows per recording.

GRU(6,16) + Linear(16,4): 1,220 parameters.

Adam; 30 epochs; validation selects checkpoint.

LSTM comparison is analytical only.

The generator is deliberately learnable; it is not a validated wrist-motion simulator.

Software demonstration and actual results

ACTUAL SAVED OUTPUT | SYNTHETIC DATA ONLY

TRUE CLASS	PREDICTED	CONFIDENCE	COMMAND
background	background	0.995	NO_COMMAND
swipe_left	swipe_left	0.992	LIGHT_OFF
swipe_right	swipe_right	0.993	LIGHT_ON
wave	wave	0.995	FAN_TOGGLE

100.0%
Synthetic test accuracy

1.000
Synthetic macro-F1

0.668 ms
Host-only median inference

Perfect toy-data scores verify this generator/pipeline only; real-world accuracy is unknown.

Limitations and future work

Evidence limits

Synthetic recordings do not cover user variation, tremor, mounting changes, or real incidental motion.

No hardware claims

MCU latency, peak RAM, flash footprint, power, and battery life have not been measured.

Future evaluation: user recordings and on-target profiling.

Next steps, if extended

- Real-data and user evaluation
- Continuous rejection + cooldown
- MCU/runtime operator checks
- Optional 8-bit quantization
- Optional wireless integration

Conclusion and reproducible demonstration

A small, working software pipeline advances the synopsis without overstating what has been built.

Implemented: six-channel synthetic input, preprocessing, windows, trained GRU, rejection, and command output.

Verified: held-out synthetic metrics and seven pipeline tests.

Run: python demo/demo.py

Reproduce: python demo/demo.py --train

Real-user performance, MCU deployment, quantization, and physical control remain future work.